

# AUTOM-05: Dynatrace Workflows

**Series:** AUTOM — Dynatrace Automation | **Notebook:** 5 of 9 | **Created:** January 2026 | **Last Updated:** 09/18/2026

Dynatrace Workflows is a built-in automation engine that enables event-driven actions directly within the platform. Unlike external tools, workflows run inside Dynatrace with full access to observability data.

## Table of Contents

- 1. [Introduction](#)
- 2. [Workflow Components](#)
- 3. [Creating Workflows](#)
- 4. [Actions and Integrations](#)
- 5. [Auto-Remediation Patterns](#)
- 6. [Best Practices](#)
- 7. [Recent Enhancements](#)
- 8. [Next Steps](#)
- 9. [Next Steps](#)
- 10. [Next Steps](#)

## Prerequisites

Before starting this notebook, ensure you have:

| Requirement    | Description                   |
|----------------|-------------------------------|
| Dynatrace SaaS | Tenant with Workflows enabled |

| Requirement | Description                               |
|-------------|-------------------------------------------|
| Permissions | AutomationWorkflows permission            |
| Basic DQL   | Understanding of Dynatrace Query Language |

## Learning Objectives

By the end of this notebook, you will:

- Understand Dynatrace Workflows architecture
- Know how to create event-driven automations
- Be able to implement auto-remediation patterns
- Connect workflows to external systems

## 1. Introduction

### Why Workflows?

| Benefit            | Description                                   |
|--------------------|-----------------------------------------------|
| Native Integration | Direct access to all Dynatrace data           |
| No Infrastructure  | No external systems to manage                 |
| Event-Driven       | React to problems, alerts, schedules          |
| Context-Aware      | Full Dynatrace Intelligence context available |
| Secure             | Credentials stored in Dynatrace vault         |

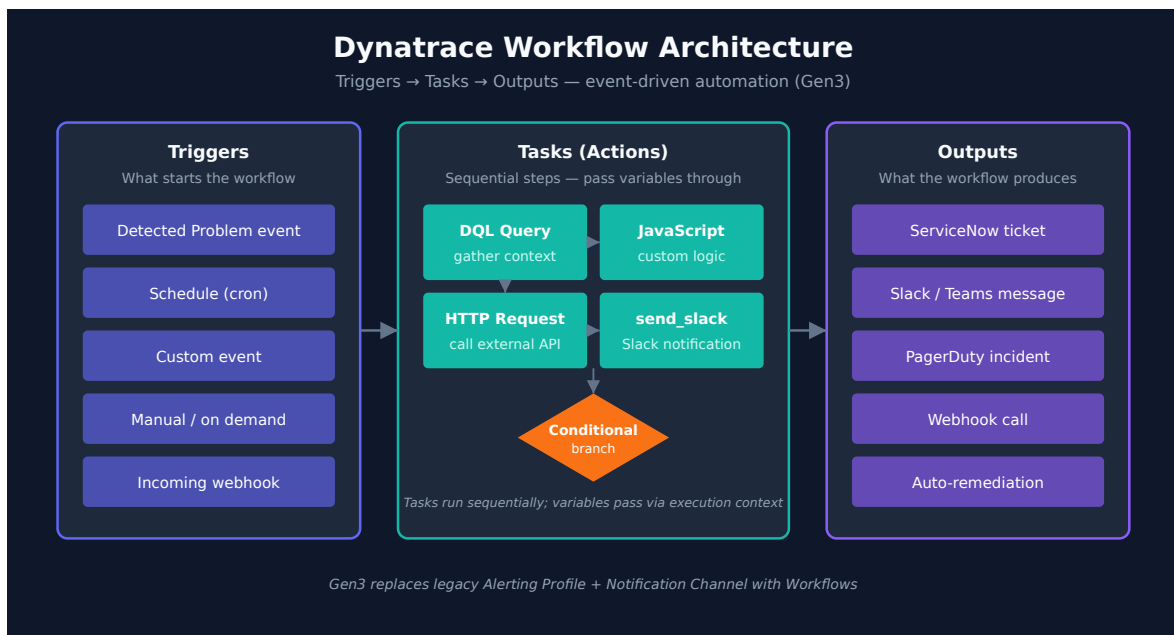
### Common Use Cases

| Use Case      | Description                        |
|---------------|------------------------------------|
| Notifications | Send alerts to Slack, Teams, email |

| Use Case         | Description                       |
|------------------|-----------------------------------|
| Auto-Remediation | Restart services, scale resources |
| Ticketing        | Create ServiceNow, Jira tickets   |
| Enrichment       | Add context to problems           |
| Reporting        | Generate scheduled reports        |

## 2. Workflow Components

### Workflow Architecture



### Triggers

| Trigger Type | Description            | Example                     |
|--------------|------------------------|-----------------------------|
| Problem      | detected problem event | Availability issue detected |
| Event        | Custom or ingest event | Deployment completed        |
| Schedule     | Time-based (cron)      | Daily at 9 AM               |
| Manual       | User-initiated         | On-demand execution         |

| Trigger Type | Description     | Example                 |
|--------------|-----------------|-------------------------|
| On Demand    | API or SDK call | External system trigger |

## Tasks

| Task Type         | Description          |
|-------------------|----------------------|
| HTTP Request      | Call external APIs   |
| Run JavaScript    | Custom logic         |
| DQL Query         | Query Grail data     |
| Send Notification | Slack, Teams, email  |
| Create Issue      | Jira, ServiceNow     |
| Run Workflow      | Call other workflows |

## 3. Creating Workflows

### Access Workflows

Navigate to: **Apps** → **Workflows**

Or via URL:

<https://{tenant}.apps.dynatrace.com/platform/app/dynatrace.automations/workflows>

### Simple Notification Workflow

**Workflow YAML (schematic — shows the trigger → task → input shape, not the exact export format):**

**Check connector action IDs in your tenant.** The Slack and ServiceNow action IDs in this notebook are illustrative. Before copying one, add the action in the workflow editor (or export an existing workflow) and use the action ID it shows. The core automation actions are `dynatrace.automations:execute-dql-query` ,

```
dynatrace.automations:run-javascript and dynatrace.automations:http-  
function .
```

```
title: Problem Notification  
description: Send Slack notification for critical problems  
trigger:  
  type: davis-problem  
  config:  
    categories:  
      - AVAILABILITY  
      - ERROR  
tasks:  
  send_slack:  
    name: Send Slack Alert  
    action: dynatrace.slack:send-message  
    input:  
      connection: slack-webhook  
      channel: "#alerts"  
      message: |  
        :warning: *Problem Detected*  
        *Problem:* {{ event()["display_id"] }} – {{ event()["event.name"] }}  
        *Category:* {{ event()["event.category"] }}  
        *Status:* {{ event()["event.status"] }}
```

Problem-trigger event fields are the Grail problem-record field names ( `event.name` , `event.category` , `event.status` , `display_id` , `root_cause_entity_id` , `affected_entity_ids` ) — the same fields `fetch dt.davis.problems` returns — not the Classic problem-API names ( `title` , `severity` , `impactLevel` , `rootCauseEntity` ).

## Problem Trigger Configuration

| Option           | Description                                     |
|------------------|-------------------------------------------------|
| Categories       | AVAILABILITY, ERROR, SLOWDOWN, RESOURCE, CUSTOM |
| Severities       | Filter by severity level                        |
| Management Zones | Scope to specific zones                         |
| Entity Types     | Filter by entity (HOST, SERVICE, etc.)          |

## Using JavaScript Tasks

Custom logic with JavaScript:

```
// Task: Enrich problem data
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function ({ execution_id }) {
  // The triggering event is on the execution, not a global
  const ex = await execution(execution_id);
  const problem = ex.event() ?? {};

  // Build enrichment data
  return {
    problemId: problem['display_id'],
    name: problem['event.name'],
    affectedEntities: (problem['affected_entity_ids'] ?? []).length,
    rootCauseEntity: problem['root_cause_entity_name'] || 'Unknown',
    timestamp: new Date().toISOString()
  };
}
```

A predecessor task's output is `await ex.result('task_name')` (or the standalone `result('task_name')` export) — see the [automation-utils SDK reference \(Dynatrace Developer\)](#).

## Using DQL Tasks

Query data within workflows:

```
tasks:
  get_metrics:
    name: Get CPU Metrics
    action: dynatrace.automations:execute-dql-query
    input:
      query: |
        timeseries avg(dt.host.cpu.usage), by:{dt.entity.host}, from:-30m
        | filter dt.entity.host == "{ event()['root_cause_entity_id'] }"
        | limit 10
```

## 4. Actions and Integrations

---

### Built-in Connectors

| Connector       | Actions                       |
|-----------------|-------------------------------|
| Slack           | Send message, post to channel |
| Microsoft Teams | Send adaptive card            |
| Jira            | Create/update issue           |
| ServiceNow      | Create incident, update CI    |
| PagerDuty       | Create incident               |
| OpsGenie        | Create alert                  |
| Email           | Send email notification       |

### HTTP Request Task

Call any REST API:

```
tasks:
  call_api:
    name: Call External API
    action: dynatrace.automations:http-function
    input:
      url: "https://api.example.com/incidents"
      method: POST
      headers:
        Content-Type: application/json
        Authorization: "Bearer {{ env.API_TOKEN }}"
      body: |
        {
          "title": "{{ event()['event.name'] }}",
          "category": "{{ event()['event.category'] }}",
          "source": "dynatrace"
        }
```

## Credential Management

Store credentials securely:

1. Go to **Settings** → **Credential Vault**
  2. Create a new credential
  3. Reference in workflow: `{{ credential.MY_API_KEY }}`
-

## 5. Auto-Remediation Patterns

---

### Pattern: Restart Service

```
title: Auto-Restart on Crash
trigger:
  type: davis-problem
  config:
    categories:
      - AVAILABILITY
tasks:
  check_entity:
    name: Validate Entity Type
    action: dynatrace.automations:run-javascript
    input:
      script: |
        import { execution } from '@dynatrace-sdk/automation-utils';
        export default async function ({ execution_id }) {
          const ex = await execution(execution_id);
          const entityId = (ex.event() ?? {})[ 'root_cause_entity_id' ] ?? '';
          if (!entityId.startsWith('PROCESS_GROUP_INSTANCE-')) {
            return { skip: true };
          }
          return { entityId, skip: false };
        }

  restart_service:
    name: Execute Restart
    action: dynatrace.automations:http-function
    conditions:
      states:
        check_entity: OK
      custom: "{{ result('check_entity').skip == false }}"
    input:
      url: "https://runbook.example.com/restart"
      method: POST
      body: |
        {
          "entityId": "{{ result('check_entity').entityId }}"
        }
```

## Pattern: Scale on High CPU

```
title: Auto-Scale on Resource Pressure
trigger:
  type: davis-problem
  config:
    categories:
      - RESOURCE
tasks:
  check_cpu:
    name: Query Current CPU
    action: dynatrace.automations:execute-dql-query
    input:
      query: |
        timeseries cpu = avg(dt.host.cpu.usage), from:-30m, filter:
{dt.entity.host == "{{ event()['root_cause_entity_id'] }}" }
        | fieldsAdd avg_cpu = arrayAvg(cpu)

  scale_up:
    name: Trigger Scale Event
    action: dynatrace.automations:http-function
    conditions:
      custom: "{{ result('check_cpu').records[0].avg_cpu > 90 }}"
    input:
      url: "https://api.cloud.example.com/scale"
      method: POST
```

A bare `timeseries avg(dt.host.cpu.usage) | filter dt.entity.host == ...` fails with `FIELD_DOES_NOT_EXIST`: without `by:{dt.entity.host}` the series carries no host dimension to filter on. Filter inside `timeseries` (as above) or add the `by:` clause.

## Pattern: Create Incident Ticket

```
title: ServiceNow Incident Creation
trigger:
  type: davis-problem
  config:
    categories:
      - AVAILABILITY
      - ERROR
tasks:
  create_incident:
    name: Create ServiceNow Incident
    action: dynatrace.servicenow:create-incident
    input:
      connection: servicenow-prod
      short_description: "Dynatrace: {{ event()['event.name'] }}"
      description: |
        Problem detected by Dynatrace Intelligence

        Problem: {{ event()['display_id'] }}
        Name: {{ event()['event.name'] }}
        Category: {{ event()['event.category'] }}
        Root Cause: {{ event()['root_cause_entity_name'] }}
      urgency: 2
      impact: 2

  add_comment:
    name: Comment on Problem
    action: dynatrace.automations:run-javascript
    input:
      script: |
        import { result } from '@dynatrace-sdk/automation-utils';
        export default async function () {
          const incident = await result('create_incident');
          // Add comment to Dynatrace problem with ticket reference
          return { ticketId: incident?.sys_id, status: 'created' };
        }
```

---

## 6. Best Practices

---

### Workflow Design

| Practice              | Description                     |
|-----------------------|---------------------------------|
| Single responsibility | One workflow, one purpose       |
| Idempotency           | Safe to run multiple times      |
| Error handling        | Handle failures gracefully      |
| Logging               | Add descriptive task names      |
| Testing               | Use manual triggers for testing |

### Trigger Configuration

| Practice         | Description                      |
|------------------|----------------------------------|
| Scope narrowly   | Use management zones and filters |
| Avoid duplicates | Check if action already taken    |
| Debounce         | Add delays for flapping alerts   |

### Security

| Practice         | Description                 |
|------------------|-----------------------------|
| Credential vault | Never hardcode secrets      |
| Least privilege  | Minimal API permissions     |
| Audit trail      | Log all remediation actions |

### Error Handling

Workflows have no `on_error` or `error()` construct. Retries are a task property ( `count` 1–99, `delay` in **seconds** 1–3600), and error routing is a downstream task whose **condition** is the upstream task's state ( `SUCCESS` , `ERROR` , `ANY` , `OK` , `NOK` ):

```

tasks:
  api_call:
    name: Call API
    action: dynatrace.automations:http-function
    retry:
      count: 3
      delay: 30          # seconds

  error_handler:
    name: Handle Error
    action: dynatrace.slack:send-message  # verify the action ID in your
workflow editor
    conditions:
      states:
        api_call: ERROR
    input:
      channel: "#alerts"
      message: "Task api_call failed after 3 retries"

```

The same fields in Terraform form are documented on the [dynatrace\\_automation\\_workflow resource \(Dynatrace GitHub\)](#) — `retry { count, delay, failed_loop_iterations_only }` and `conditions { states = { ... } }`.

## 7. Recent Enhancements

### New Workflow Features (2025-2026)

Dynatrace has added several capabilities to the Workflows engine:

| Feature                  | Description                                                                                |
|--------------------------|--------------------------------------------------------------------------------------------|
| <b>Sub-workflows</b>     | Call reusable workflows from within other workflows for modular automation                 |
| <b>Approval Requests</b> | Built-in human approval gates for governed automation                                      |
| <b>Workflow Drafts</b>   | Save work-in-progress workflows without deploying them                                     |
| <b>Simple Workflows</b>  | Quickly automate single-step tasks (e.g., send a Slack notification) at no additional cost |

| Feature                   | Description                                                       |
|---------------------------|-------------------------------------------------------------------|
| Persistent Execution Data | Retain execution history for analysis and refinement              |
| Real-time Notifications   | Get notified when workflows complete or fail                      |
| ServiceNow Integration    | Six pre-built integrations for incident creation, CMDB enrichment |

## Dynatrace Intelligence Agents

With Dynatrace Intelligence (announced Perform 2026), workflows can now host **agentic workflows** that use Intelligence Agents for autonomous operations:

| Capability              | Description                                                 |
|-------------------------|-------------------------------------------------------------|
| Agentic Workflows       | AI agents execute decisions within workflow tasks           |
| Closed-loop Remediation | Agents can autonomously resolve issues with guardrails      |
| SRE Agents              | Specialized agents for incident triage and resolution       |
| Governance              | Human oversight controls for supervised or autonomous modes |

**Note:** Intelligence Agents build on top of the Workflows engine. Existing workflow patterns remain valid; Agents add an AI reasoning layer for autonomous decision-making.

## 8. Next Steps

### Workflows vs External Automation

| Scenario                    | Tool Choice      |
|-----------------------------|------------------|
| Event-driven from Dynatrace | Workflows        |
| Config deployment           | Monaco/Terraform |

| Scenario               | Tool Choice               |
|------------------------|---------------------------|
| Complex orchestration  | External (Ansible, etc.)  |
| Cross-platform actions | Workflows with HTTP tasks |

## Continue the Series

| Next Notebook                   | Focus                                      |
|---------------------------------|--------------------------------------------|
| <b>AUTOM-06: Dynatrace SDKs</b> | Programmatic access with TypeScript/Python |

## Additional Resources

- [Workflows Documentation](#)
- [Workflow Actions Reference](#)

---

## Summary

In this notebook, you learned:

- Dynatrace Workflows architecture and components
- Creating event-driven automations with triggers and tasks
- Implementing auto-remediation patterns
- Best practices for workflow design and security

**Key Takeaway:** Workflows provide native automation inside Dynatrace. Use them for event-driven actions like notifications, ticketing, and auto-remediation. For configuration management, continue using Monaco or Terraform.

---

*Continue to AUTOM-06: Dynatrace SDKs to learn programmatic access patterns.*

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*